

Lesson:



Backtracking 2



Prerequisite:

- Backtracking

Today's Checklist:

- Problems on Backtracking

Problem 1:Check Knight Tour Configuration

There is a knight on an $n \times n$ chessboard. In a valid configuration, the knight starts at the top-left cell of the board and visits every cell on the board exactly once.

You are given an $n \times n$ integer matrix grid consisting of distinct integers from the range $[0, n * n - 1]$ where $\text{grid}[\text{row}][\text{col}]$ indicates that the cell (row, col) is the $\text{grid}[\text{row}][\text{col}]$ th cell that the knight visited. The moves are 0-indexed.

Return true if grid represents a valid configuration of the knight's movements or false otherwise.

Note that a valid knight move consists of moving two squares vertically and one square horizontally, or two squares horizontally and one square vertically. The figure below illustrates all the possible eight moves of a knight from some cell.

Input:

0	11	16	5	20
17	4	19	10	15
12	1	8	21	6
3	18	23	14	9
24	13	2	7	22

Output:

true

Efficient Approach:

- This problem can be solved using backtracking. We tested all the possible moves from the current index. If the result of the current cell is one value bigger than the previous cell, then it is valid to go there.
- For each valid move, we decrement the countMoves variable by 1.
- If $\text{countValid} == 0$, means that we tried all the possible moves, return true.

[Code Link](#)

Time Complexity –

Time complexity will be $O(n * n)$.

Space Complexity –

Space complexity will be $O(n * n)$.

Problem 2: Place K-knights such that they do not attack each other

Given integers M, N and K, the task is to place K knights on an $M * N$ chessboard such that they don't attack each other. The knights are expected to be placed on different squares on the board. A knight can move two squares vertically and one square horizontally or two squares horizontally and one square vertically. The knights attack each other if one of them can reach the other in single move. There are multiple ways of placing K knights on

an $M \times N$ board or sometimes, no way of placing them. We are expected to list out all the possible solutions.

Examples:

Input: $M = 3, N = 3, K = 5$

Output: K A K A K A K A K A K K K A K A

Total number of solutions : 2



Input:

$M = 5, N = 5, K = 13$

Output:

K A K A K A K A K A K A K A K A K A K

Total number of solutions : 1

Approach:

- This problem can be solved using backtracking. The idea is to place the knights one by one starting from first row and first column and moving forward to first row and second column such that they don't attack each other.
- When one row gets over, we move to the next row. Before placing a knight, we always check if the block is safe i.e. it is not an attacking position of some other knight.
- If it is safe, we place the knight and mark it's attacking position on the board else we move forward and check for other blocks.
- While following this procedure, we make a new board every time we insert a new knight into our board. This is done because if we get one solution and we need other solutions, then we can backtrack to our old board with old configuration of knights which can then be checked for other possible solutions.
- The process of backtracking is continued till we get all our possible solutions.

Below is the implementation of the above approach:

[Code Link](#)

Problem 3: Maximum non-attacking Knights that can be placed on an $N \times M$ Chessboard

Given an $N \times M$ chessboard. The task is to find the maximum number of knights that can be placed on the given chessboard such that no knight attack some other knight.

Input: $N = 1, M = 4$

Output: 4Place a knight on every cell of the chessboard.

Input2: N = 4, M = 5
Output2: 10

Efficient Approach:

- The approach takes into account different scenarios to determine the maximum number of knights based on the dimensions of the chessboard.
- For the corner cases where either n or m is equal to 1, it is straightforward. Since a single row or column cannot have multiple knights attacking each other, the maximum number of knights will be equal to the maximum of n and m.
- For the corner case where either n or m is equal to 2, the code calculates the maximum number of knights by considering consecutive 2 rows or columns. It divides the maximum of n and m by 4 and multiplies it by 4 to get the number of knights that can be placed. If there is a remainder, it adjusts the count accordingly. If the remainder is 1, it adds 2 knights because they can be placed in a way that they don't attack each other. If the remainder is greater than 1, it adds 4 knights because they can be placed in a way that they don't attack each other.
- For the general case where n and m are greater than 2, the code simply calculates the half of the total number of blocks (n * m) on the chessboard. This is because each knight occupies two blocks on the chessboard, and placing them optimally would require filling half of the available blocks.
- The approach uses simple arithmetic calculations to determine the maximum number of knights in each scenario. There are no loops or recursive calls involved, which leads to a constant time complexity.

Code link

Time Complexity –
Time complexity will be O(1).
Space Complexity –
Space complexity will be O(1).

Output:

```
"C:\Program Files\Java\jdk-14.0.1\  
10  
  
Process finished with exit code 0
```

Problem 4: Valid Sudoku

Determine if a 9 x 9 Sudoku board is valid. Only the filled cells need to be validated according to the following rules:
Each row must contain the digits 1-9 without repetition.
Each column must contain the digits 1-9 without repetition.
Each of the nine 3 x 3 sub-boxes of the grid must contain the digits 1-9 without repetition.

Input:

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Output: true

Efficient Approach:

- The `isSafe` method is used to check if it is safe to place a digit at a given position (r, c) on the board. It checks the row, column, and box to ensure that the digit is not already present in any of them. If the digit is found in any of these places, the method returns false, indicating that it is not safe to place the digit at that position.
- The `isValid` method is a recursive function that uses backtracking to solve the Sudoku puzzle. It starts by checking if the current column index c is equal to the length of a row (indicating the end of a row). If so, it moves to the next row by incrementing r and setting c to 0. This is the base condition that signifies the completion of the entire board, in which case the method returns true.
- If the current cell contains a digit (not equal to '.'), the method temporarily replaces it with '.', checks if it is safe to place the digit at that position using the `isSafe` method, and then backtracks by restoring the original digit. This is done to explore all possible valid solutions for the Sudoku puzzle.
- Finally, the `isValidSudoku` method is a wrapper function that calls the `isValid` method with initial values of 0 for r and c . It returns the result of the `isValid` method, indicating whether the given Sudoku board is a valid solution.

Time Complexity –

Time complexity will be $O(n*n)$.

Space Complexity –

Space complexity will be $O(1)$.

[Code link](#)

Problem 5: Sudoku Solver ()

Consider a $9*9$ 2D array grid that is partially filled with numbers from 1 to 9. The Sudoku Solver problem is to fill remaining blocks with numbers from 1 to 9 so that every row, column and subgrid ($3*3$) contains exactly one instance of digits (1 to 9).

Example:Input – (Unfilled cells are denoted as 0).

```
{ {3, 0, 6, 5, 0, 8, 4, 0, 0},
  {5, 2, 0, 0, 0, 0, 0, 0, 0},
  {0, 8, 7, 0, 0, 0, 0, 3, 1},
  {0, 0, 3, 0, 1, 0, 0, 8, 0},
  {9, 0, 0, 8, 6, 3, 0, 0, 5},
  {0, 5, 0, 0, 9, 0, 6, 0, 0},
  {1, 3, 0, 0, 0, 0, 2, 5, 0},
  {0, 0, 0, 0, 0, 0, 0, 7, 4},
  {0, 0, 5, 2, 0, 6, 3, 0, 0} }
```

Output –

```

3 1 6 5 7 8 4 9 2
5 2 9 1 3 4 7 6 8
4 8 7 6 2 9 5 3 1
2 6 3 4 1 5 9 8 7
9 7 4 8 6 3 1 2 5
8 5 1 7 9 2 6 4 3
1 3 8 9 4 7 2 5 6
6 9 2 3 5 1 8 7 4
7 4 5 2 8 6 3 1 9

```

Naive Approach – Try to generate all possible combinations and print the combination satisfying the condition.

Efficient Approach – We can solve this problem using Recursion:

- Create a function that will check if the assignment of the number to the cell is safe or not.
- Create a recursive function that will take the grid as input.
- Assign a number to an unfilled cell, after which one must check if it is safe to assign. If it is safe to assign, then call the function for all safe cases. If any recursive call returns true, simply return true. If no recursive call returns true, then return false.
- If there is no unassigned cell, return true.

Time Complexity –

Time complexity will be $O(9^{(N*N)})$.

Space Complexity –

Space complexity will be $O(N*N)$.

[Code link](#)

Output:

```

5 2 9 1 3 4 7 6 8
4 8 7 6 2 9 5 3 1
2 6 3 4 1 5 9 8 7
9 7 4 8 6 3 1 2 5
8 5 1 7 9 2 6 4 3
1 3 8 9 4 7 2 5 6
6 9 2 3 5 1 8 7 4
7 4 5 2 8 6 3 1 9

```

Upcoming Class Teasers:

- Trees